

Workshop 02 – CNN auf über 90 % Accuracy optimieren

Ziel

Erweitere den vorgegebenen Starter-Code aus Workshop 01 so, dass das Modell auf CIFAR-10 eine Genauigkeit (Accuracy) von **über 90 %** erreicht. Setze dazu verschiedene Techniken ein: eine tiefere Architektur, BatchNormalization, Dropout, Callbacks und einen Learning-Rate-Scheduler.

Datensatz

Wie in Workshop 01: **CIFAR-10** über Keras (`cifar10.load_data()`).

Ausgangspunkt

Der folgende Code (aus Workshop 01) liegt als `workshop.py` bereit und ist der Ausgangspunkt für alle Erweiterungen:

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten, Dropout
from tensorflow.keras.optimizers import Adam

# Daten laden und vorbereiten
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

# Modell erstellen
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    MaxPooling2D(2, 2),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(2, 2),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

# Modell kompilieren
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Modell trainieren
model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))

# Modell evaluieren
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f'Test Accuracy: {test_acc}')
```

Aufgaben

1. Datenvorbereitung

- Lade den CIFAR-10-Datensatz und normalisiere die Bilder.
- Teile die Daten in Trainings- und Testdaten auf.

2. Modellarchitektur

- Erstelle ein tieferes Convolutional Neural Network (CNN) mit mehr Schichten.

- Füge nach jedem Convolutional Layer eine `BatchNormalization`-Schicht hinzu.
 - Integriere `Dropout`-Schichten, um Overfitting zu verhindern.
3. **Modellkompilierung**
- Verwende den Adam-Optimizer mit einer initialen Lernrate von 0.0005.
 - Kompiliere das Modell mit der Loss-Funktion `sparse_categorical_crossentropy` und der Metrik `accuracy`.
4. **Callbacks**
- `ModelCheckpoint`: speichere das beste Modell basierend auf der Validierungsgenauigkeit.
 - `EarlyStopping`: stoppe das Training, wenn sich die Validierung nicht mehr verbessert.
 - `LearningRateScheduler`: halbiere die Lernrate nach einer bestimmten Anzahl von Epochen.
5. **Trainingsprozess**
- Trainiere das Modell für bis zu 50 Epochen mit einer Batch-Größe von 64.
 - Visualisiere den Trainingsverlauf (`accuracy` und `val_accuracy`) und speichere den Plot als `plot.png`.
6. **Vorhersagen**
- Implementiere einen Vorhersagemodus (`--predict`), der das gespeicherte Modell lädt und die Vorhersage für ein zufälliges Testbild anzeigt.

Bonus (freiwillig)

- Data Augmentation (Spiegeln, Rotieren) auf den Trainingsdaten – wie verändert sich die Test-Accuracy?
- Zwischenspeichern der vorbereiteten Daten als `.npy`, damit der Vorhersagemodus ohne erneutes Laden auskommt.

Hinweise

- `padding='same'` bei den Conv-Layern hilft, die räumliche Auflösung nicht zu schnell zu verlieren.
- Mit CPU dauert das vollständige Training deutlich länger als in Workshop 01. Teste die Pipeline zunächst mit wenigen Epochen.